896.51
Рейтинг

OTUS

Цифровые навыки от ведущих экспертов

MaxRokatansky 24 мар 2020 в 17:44

Разбираемся с launchMode Android Activity: standard, singleTop, singleTask и singleInstance

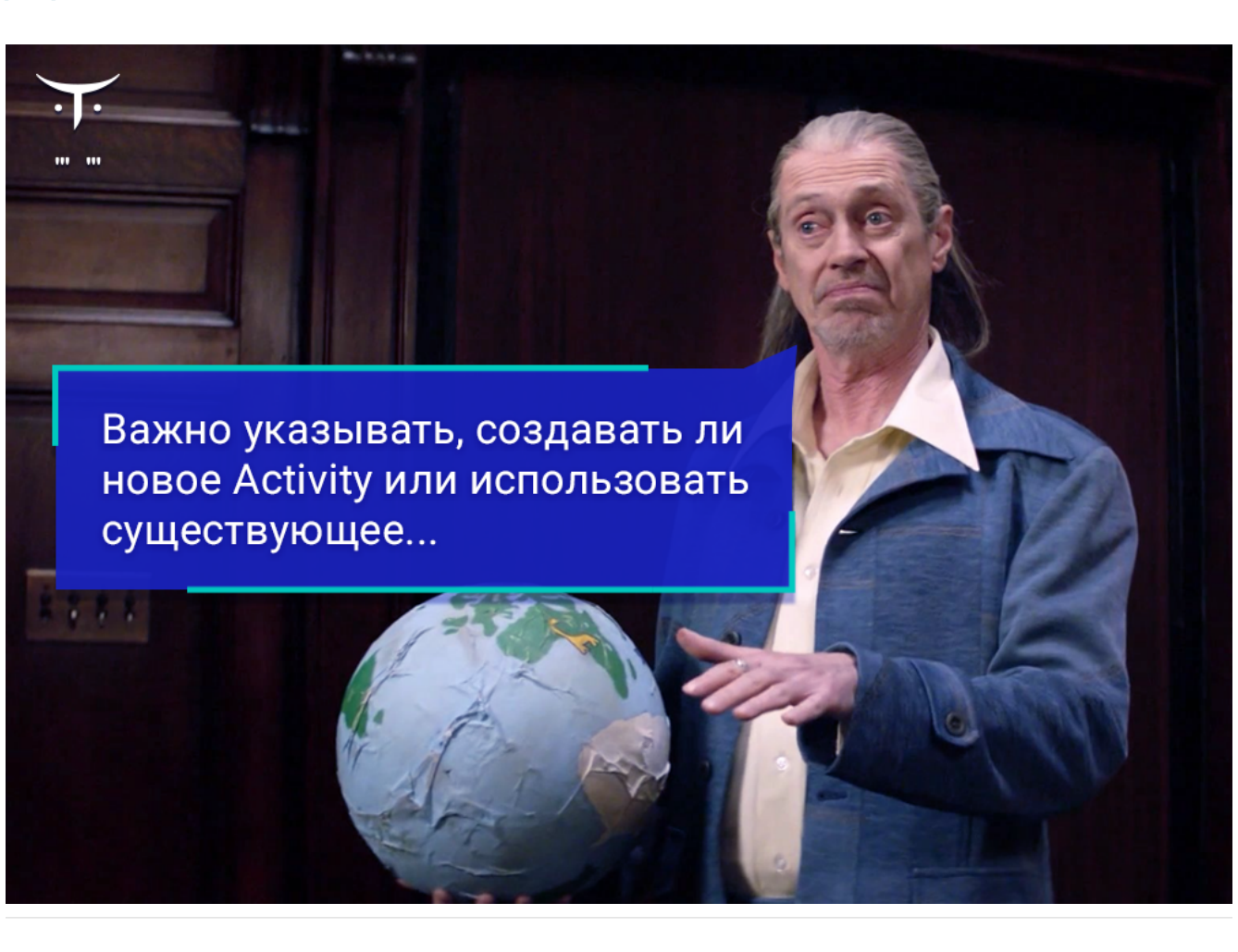
8 мин 46K

Блог компании OTUS, Разработка под Android*

Перевод

Автор оригинала: nuuneoi

Перевод статьи подготовлен специально для студентов продвинутого курса по Android разработке.


Важно указывать, создавать ли новое Activity или использовать существующее...

Activity — это одна из самых ярких концепций в Android (самой популярной мобильной операционной системе с хорошо продуманной архитектурой управления памятью, которая отлично реализует многозадачность).

Так или иначе, с запуском Activity на экран не все так однозначно. Способ, которым оно было запущено, также важен. Нюансов в этой теме очень много. Одним из действительно важных является **launchMode**, о котором мы и поговорим в этой статье.

Каждое Activity создается для работы с разными целями. Некоторые из них предназначены для работы отдельно с каждым Intent, например, отправленным Activity для составления электронной почты в почтовом клиенте. В то время как другие предназначены для работы в качестве синглтона, например, Activity почтового ящика.

Вот почему важно указывать, нужно ли создавать новое Activity или использовать существующее, иначе это может привести к плохому UX или сбоям. Благодаря разработчикам ядра Android, это легко сделать с помощью **launchMode**, который был специально для этого разработан.

Определение launchMode

По сути, мы можем определить **launchMode** напрямую в качестве атрибута тега `<activity>` в `AndroidManifest.xml`:

```
<activity
    android:name=".SingleTaskActivity"
    android:label="singleTask launchMode"
    android:launchMode="singleTask">
```

Доступно 4 типа launchMode. Давайте рассмотрим их по очереди.

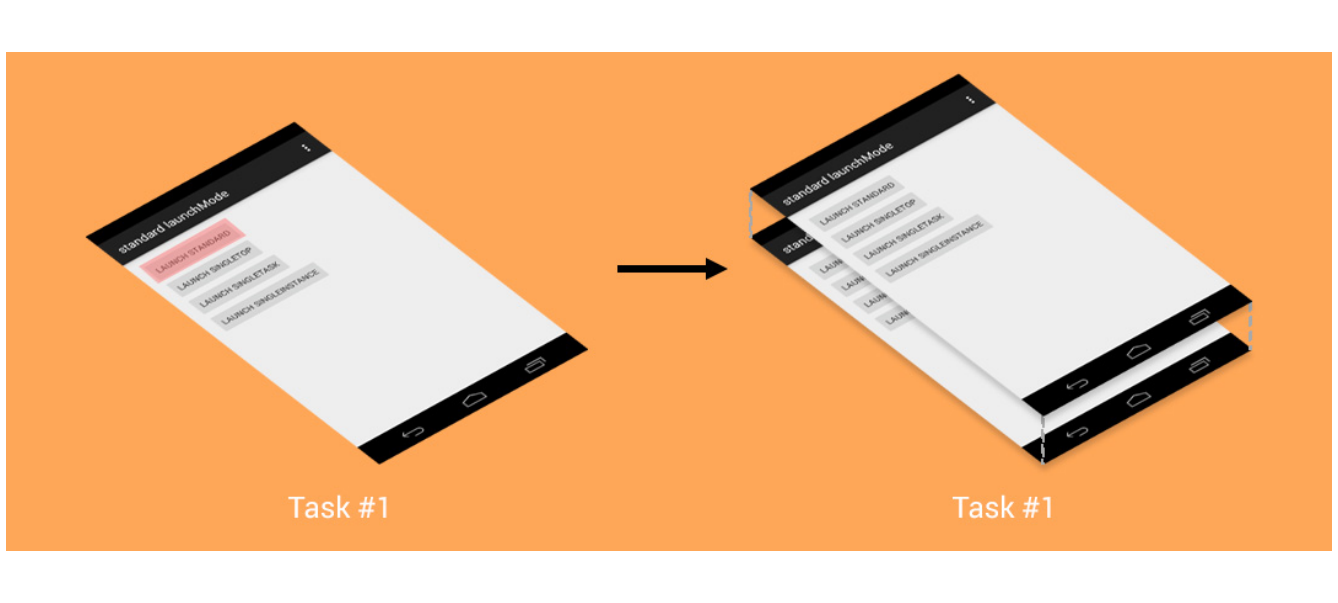
standard

Это режим «по умолчанию».

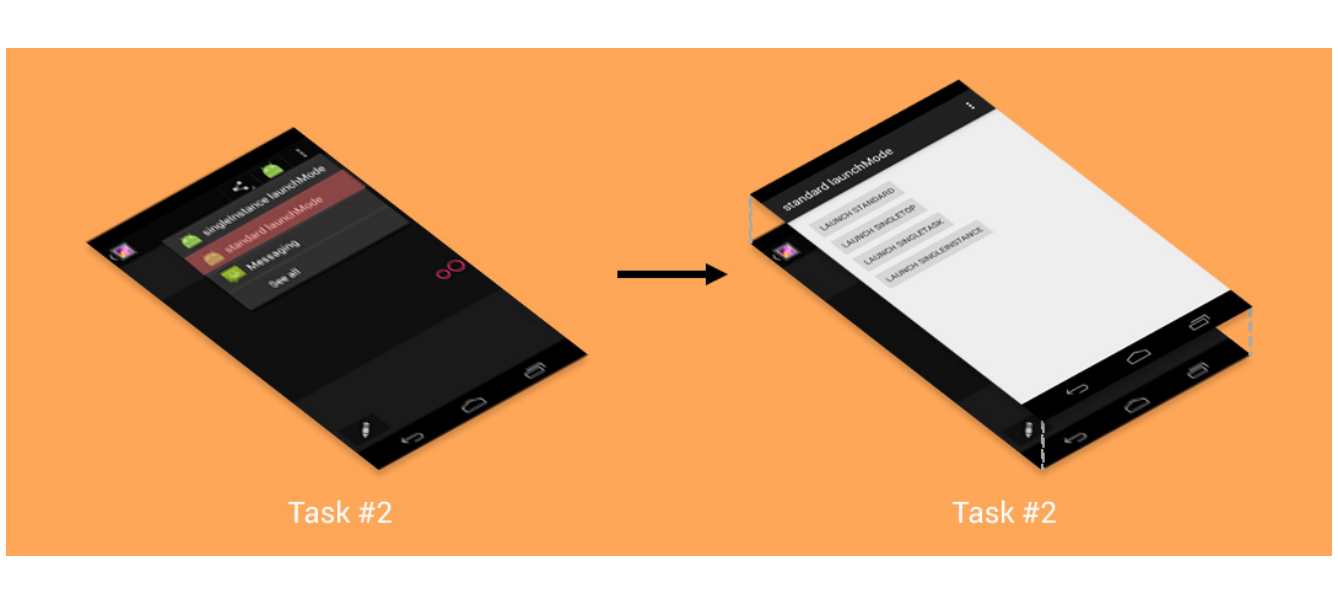
Поведение Activity, установленного в этот режим, будет всегда создавать новую Activity, чтобы работать отдельно с каждым отправленным Intent. По сути, если для составления электронного письма отправлено 10 Intent-ов, должно быть запущено 10 Activity, чтобы обслуживать каждый Intent отдельно. В результате на устройстве может быть запущено неограниченное количество таких Activity.

Поведение на пре-Lollipop Android

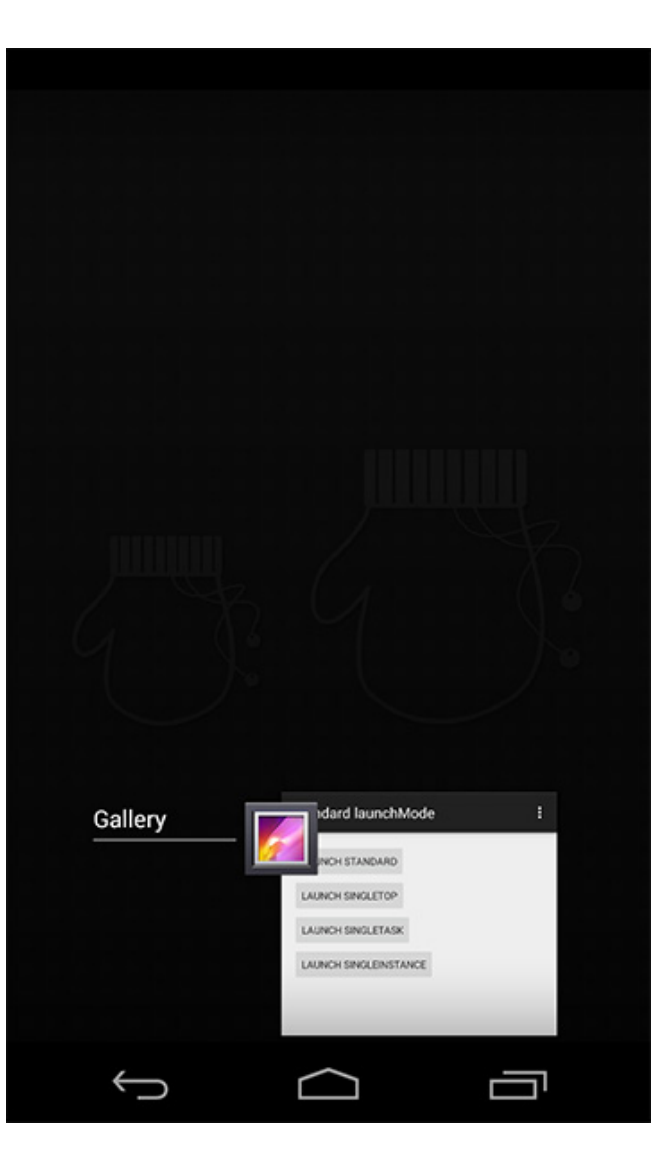
Этот вид Activity будет создан и помещен в верх стека в той же задаче, которая и отправила Intent.



На рисунке ниже показано, что произойдет, когда мы поделимся изображением со стандартным Activity. Оно будет расположено в стеке в той же задаче, как описано выше, хотя они из разных приложений.



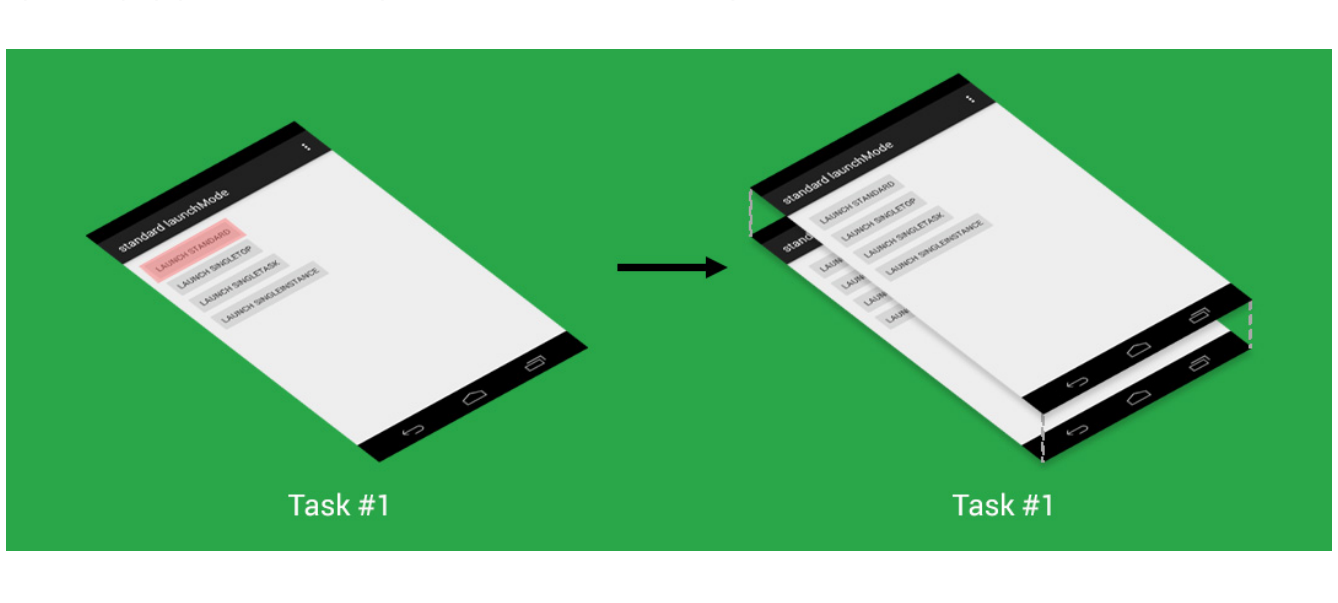
А это то, что вы увидите в диспетчере задач. (Может показаться немного странным)



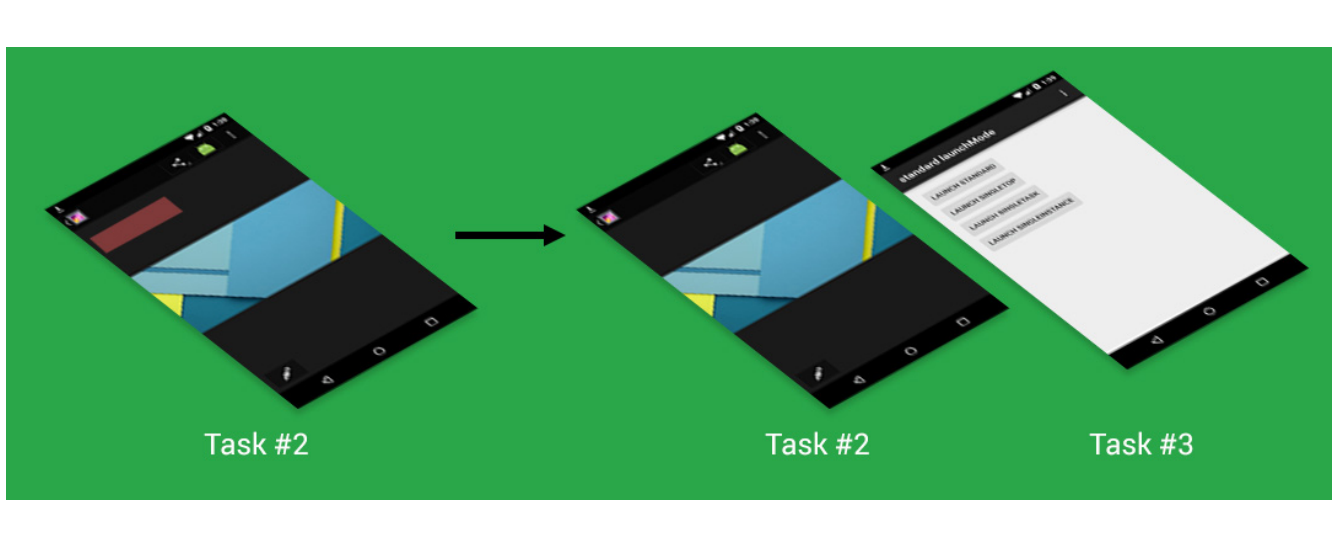
Если мы переключим приложение на другую задачу, а затем переключимся обратно в Галерею, мы все равно увидим, что стандартный launchMode помещается поверх задачи Галереи. В результате, если нам нужно что-то сделать в Галерее, мы должны сначала закончить нашу работу в этом дополнительном Activity.

Поведение на Lollipop Android

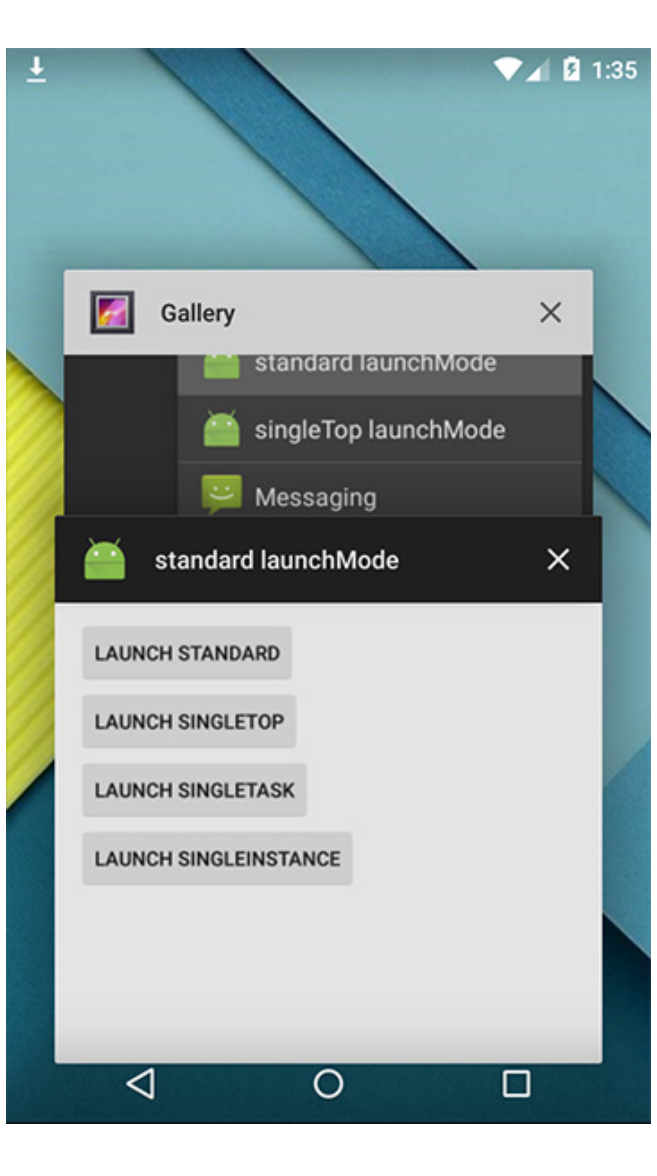
Если эти Activity относятся к одному и тому же приложению, поведение будет таким же, как и в пре-Lollipop реализации — размещение в стеке поверх задачи.



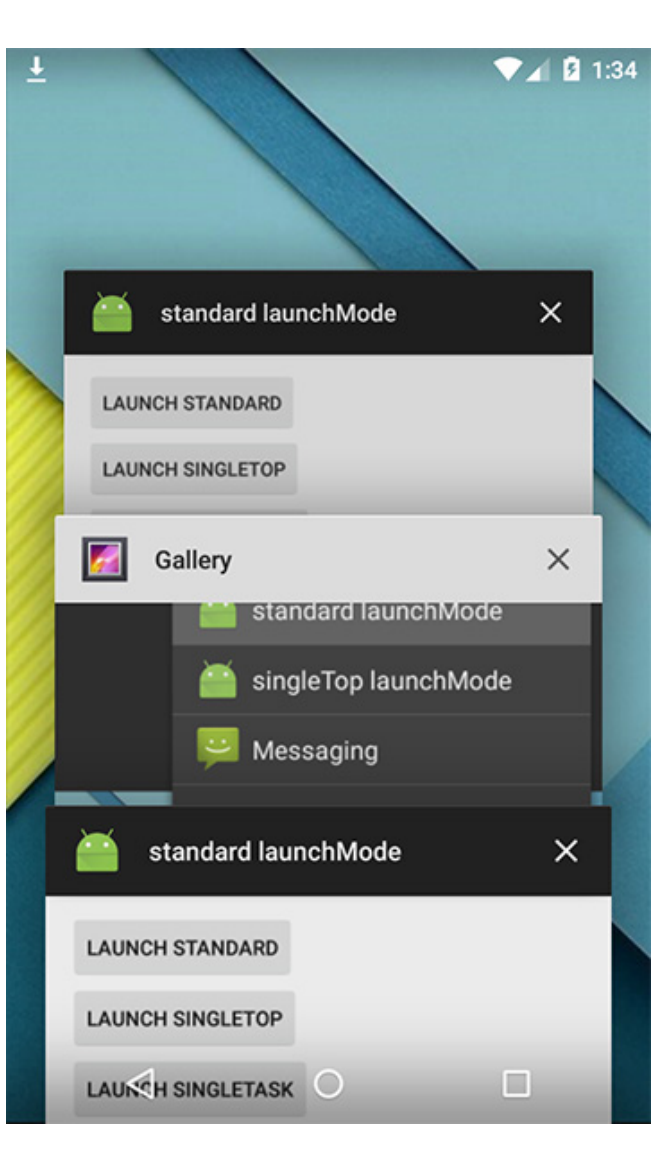
Но в случае, если Intent отправлен из другого приложения, будет создана новая задача и вновь созданное Activity будет размещено в качестве корневого, как показано ниже.



Это то, что вы увидите в диспетчере задач.



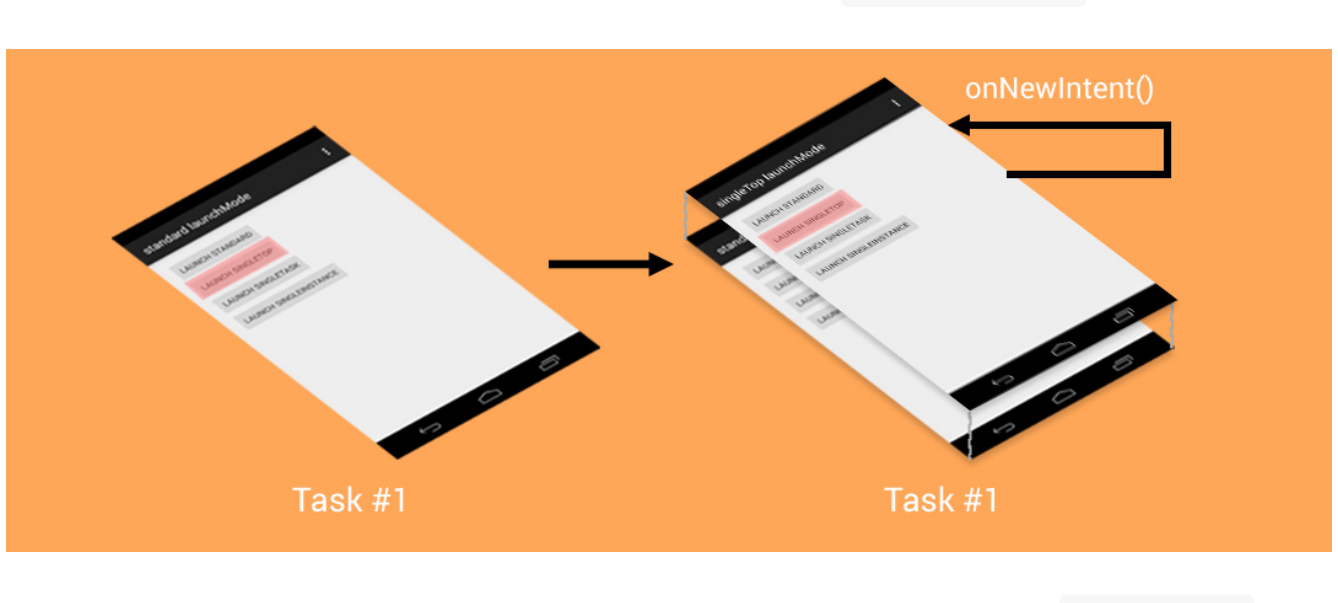
Это происходит потому, что в Lollipop модифицирована система управления задачами — она стала лучше и понятнее. В Lollipop вы можете просто переключиться обратно в Галерею, поскольку она находится в другой задаче. Вы можете отправить другой Intent, будет создана новая задача, которая будет обслуживать Intent так же, как и предыдущая.



Примером такого вида Activity является **Compose Email Activity** (составление письма) или **Social Network's Status Posting Activity** (обновление статуса в соцсети). Если у вас на уме Activity, которое отдельно обрабатывает каждый Intent, то вы думаете именно о **standard Activity**.

singleTop

Следующий режим — **singleTop**. Он ведет себя почти так же, как и **standard**, что означает, что экземпляров singleTop Activity можно создать столько, сколько мы захотим. Единственное отличие состоит в том, что если уже есть экземпляр Activity с таким же типом наверху стека в вызывающей задаче, не будет создано никакого нового Activity, вместо этого Intent будет отправлен существующему экземпляру Activity через метод `onNewIntent()`.



В режиме singleTop вы должны предусмотреть обработку входящего Intent в `onCreate()` и `onNewIntent()`, чтобы он работал во всех случаях.

Пример использования этого режима — функция поиска. Давайте подумаем о создании окна поиска, которое направляет вас к SearchActivity, чтобы увидеть результаты поиска. Для лучшего UX обычно мы всегда помещаем окно поиска на страницу результатов поиска, чтобы позволить пользователю выполнить следующий поиск, не возвращаясь назад.

А теперь представьте, что если мы всегда запускаем новое SearchActivity, чтобы обслуживать новый результат поиска, то мы получим 10 новых Activity для 10 итераций поиска. Было бы очень странно возвращаться назад, так как вам нужно было бы нажимать назад 10 раз, чтобы пройти через все результаты поиска, чтобы вернуться к корневой Activity.

Вместо этого, если SearchActivity уже находится наверху стека, лучше отправить Intent в существующий экземпляр Activity и позволить ему обновить результат поиска. Теперь будет только одно SearchActivity, размещенное наверху стека, и вы можете просто нажать кнопку «Назад» один раз, чтобы вернуться к предыдущему Activity. В этом больше смысла.

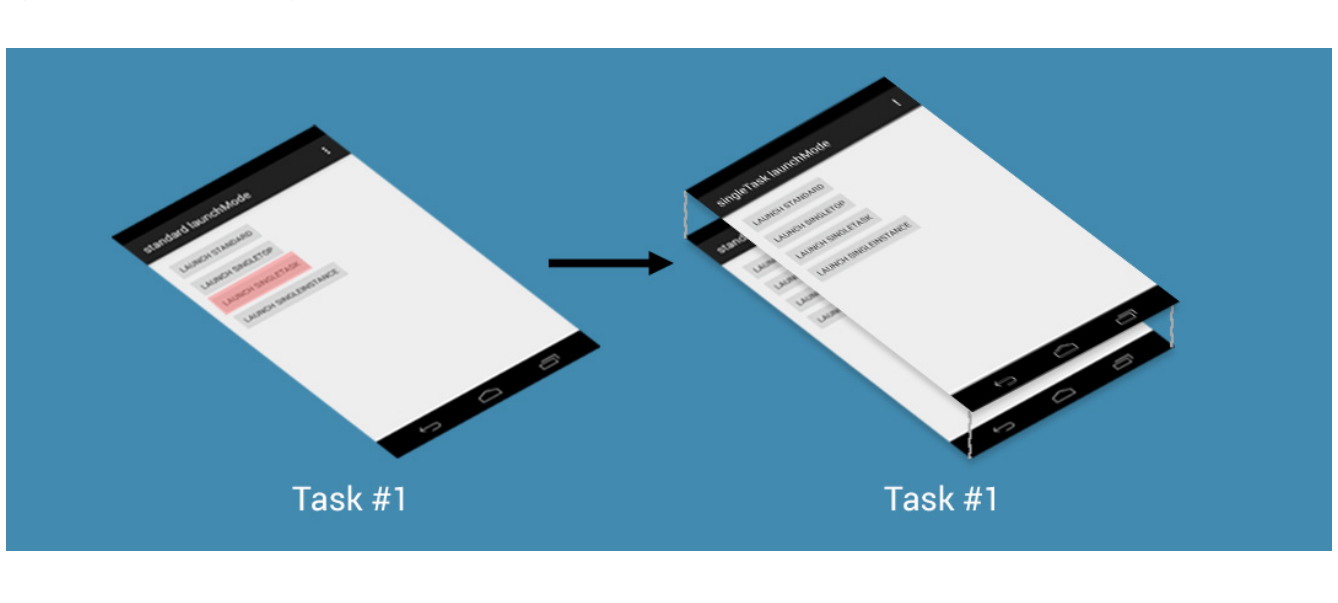
В любом случае singleTop работает в той же задаче, что и вызывающая сторона. Если вы ожидаете, что Intent будет отправлен в существующее Activity, помещенное поверх любой другой задачи, я должен вас разочаровать, сказав, что там это так уже не работает. В случае, если Intent отправлен из другого приложения в singleTop Activity, новое Activity будет запущено в том же аспекте, что и для standard launchMode (*пре-Lollipop: помещено поверх вызывающей задачи, Lollipop: будет создана новая задача*).

singleTask

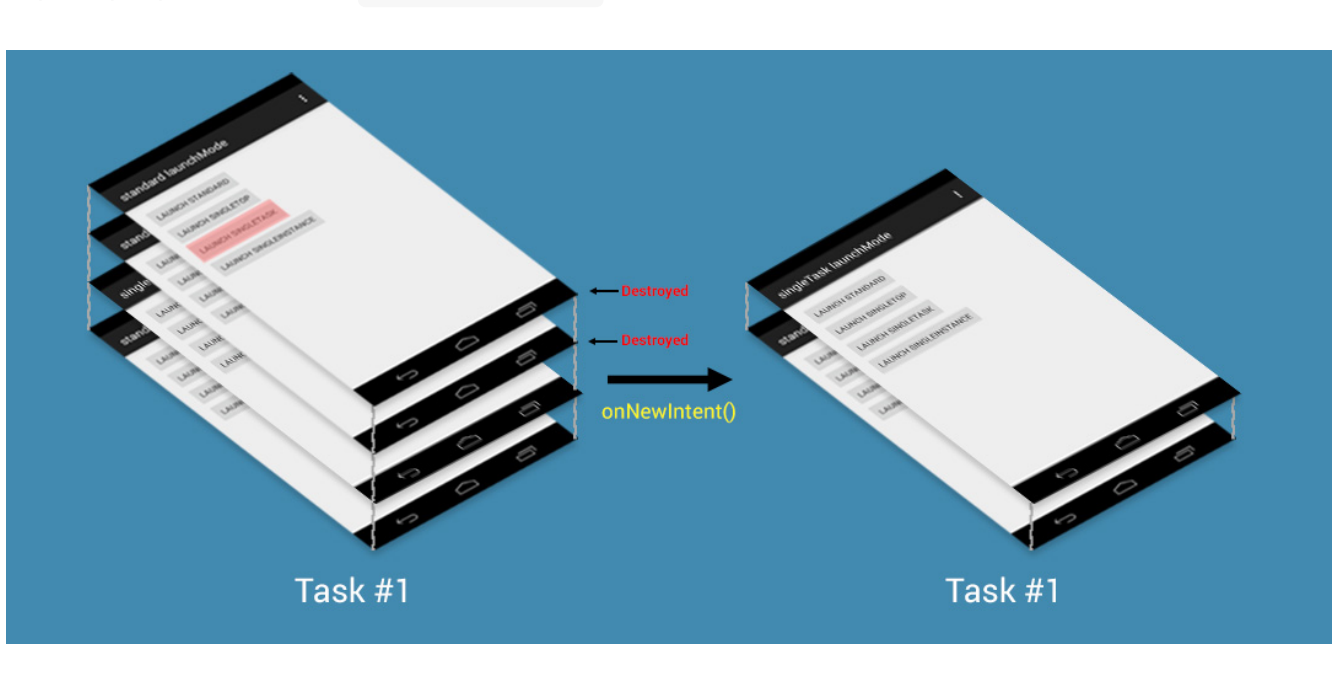
Этот режим сильно отличается от standart и singleTop. **Activity с singleTask launchMode разрешено иметь только один экземпляр в системе (аля синглтон)**. Если в системе уже существует экземпляр Activity, вся задача, удерживающая экземпляр, будет перемещен наверх, а Intent будет предоставлен через метод `onNewIntent()`. В противном случае будет создано новое Activity и помещено в соответствующую задачу.

Работая в одном приложении

Если в системе еще не было экземпляра singleTask Activity, будет создан новый, и он будет просто помещен вверх стека в той же задаче.



Но если он существует, все Activity, расположенные над этим singleTask Activity, автоматически будут жестоко уничтожены надлежащим образом (жизненный цикл закончен), чтобы отобразить на вершине стека нужное нам Activity. В то же время Intent будет отправлен в singleTask Activity через прекрасный метод `onNewIntent()`.



Это не имеет смысла с точки зрения пользовательского опыта, но оно он разработано именно таким образом…

Вы можете заметить один нюанс, который упоминается в документации:

Система создает новую задачу и инстанцирует экземпляр activity в корне новой задачи.

Но на практике похоже, что это работает не так, как описано. SingleTask Activity по-прежнему помещается наверх стека Activity задачи, как видно из результата команды `dumpsys activity`.

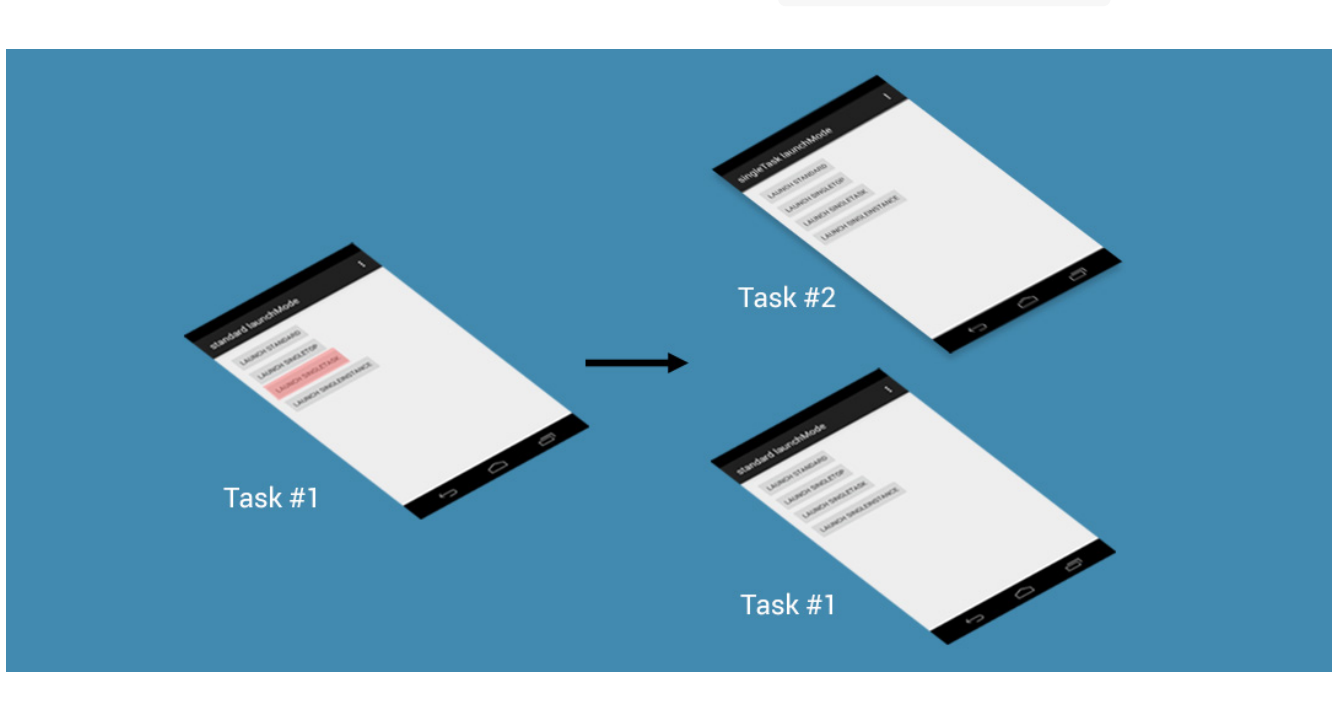
Та

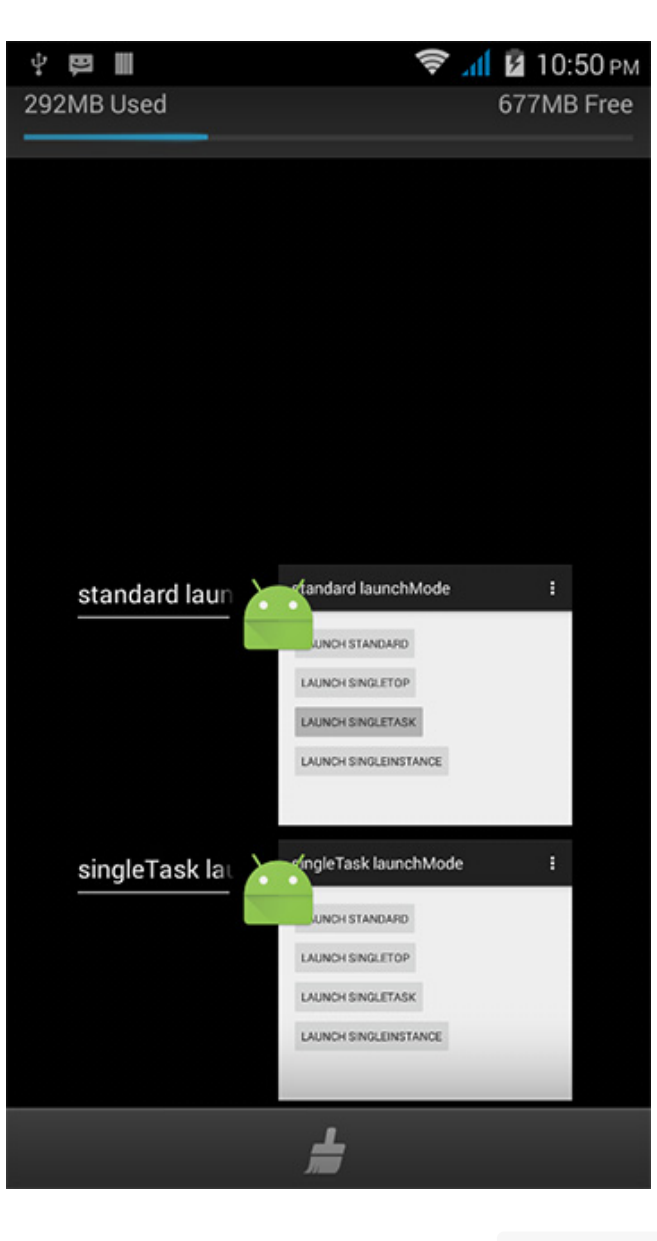
```
sk id #239
  TaskRecord{428efe30 #239 A=com.thecheesefactory.lab.launchmode U=0 sz=2}
  Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] flg=
    Hist #1: ActivityRecord{429a88d0 u0 com.thecheesefactory.lab.launchmode/.SingleTa
      Intent { cmp=com.thecheesefactory.lab.launchmode/.SingleTaskActivity }
      ProcessRecord{42243130 18965:com.thecheesefactory.lab.launchmode/u0a123}
    Hist #0: ActivityRecord{425fec98 u0 com.thecheesefactory.lab.launchmode/.Standard
      Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER]
      ProcessRecord{42243130 18965:com.thecheesefactory.lab.launchmode/u0a123}
```

Если вы хотите, чтобы singleTask Activity вело себя так, как описано в документе: создайте новую задачу и поместите Activity в качестве корневого Activity. Вам нужно определить атрибут `taskAffinity` для singleTask Activity следующим образом.

```
<activity
    android:name=".SingleTaskActivity"
    android:label="singleTask launchMode"
    android:launchMode="singleTask"
    android:taskAffinity="">
```

Таким будет результат, когда мы попытаемся запустить `SingleTaskActivity`.

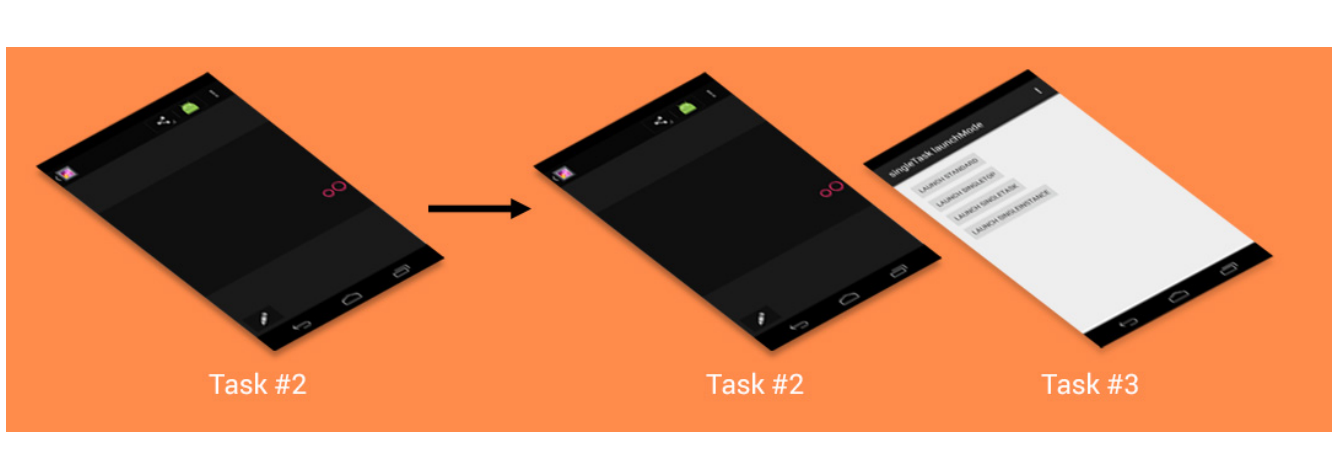


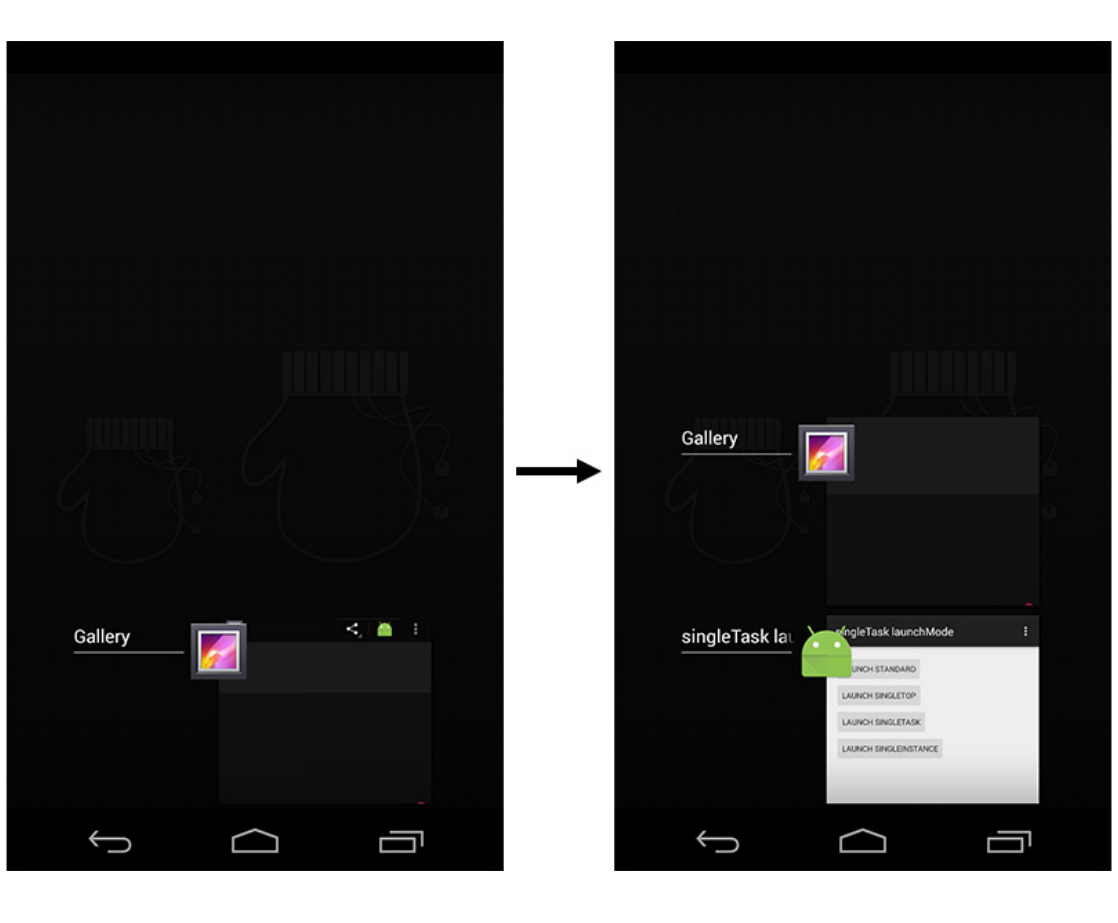


Ваша задача решить, использовать `taskAffinity` или нет в зависимости от желаемого поведения Activity.

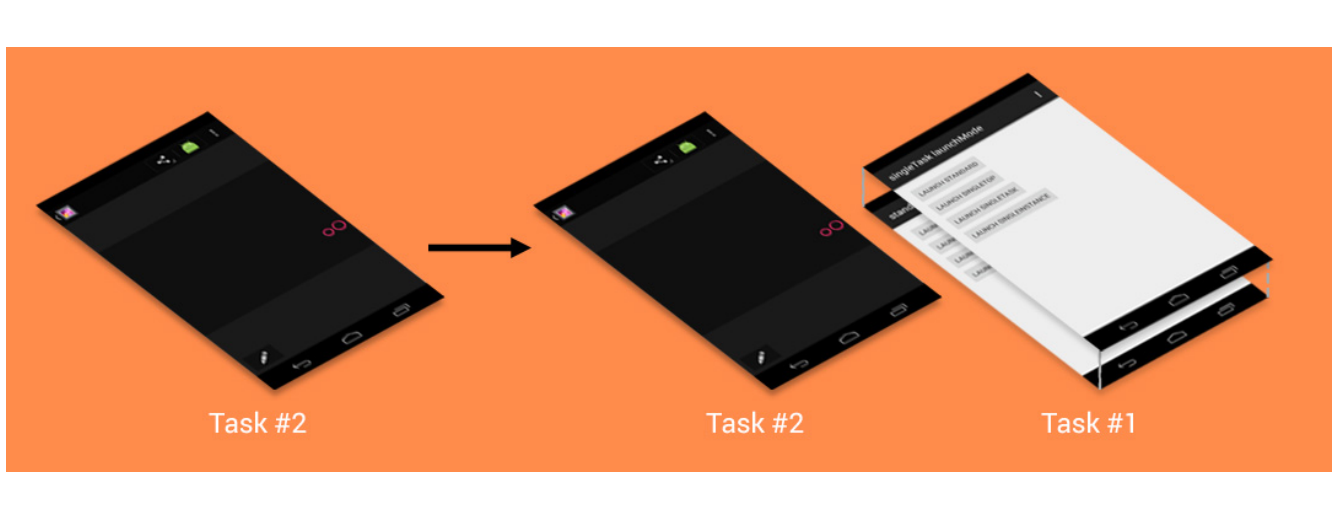
Взаимодейства с другим приложением

Как только Intent отправлен из другого приложения, и в системе еще не создано ни одного экземпляра Activity, будет создана новая задача с новым Activity, размещенным в качестве корневого Activity.

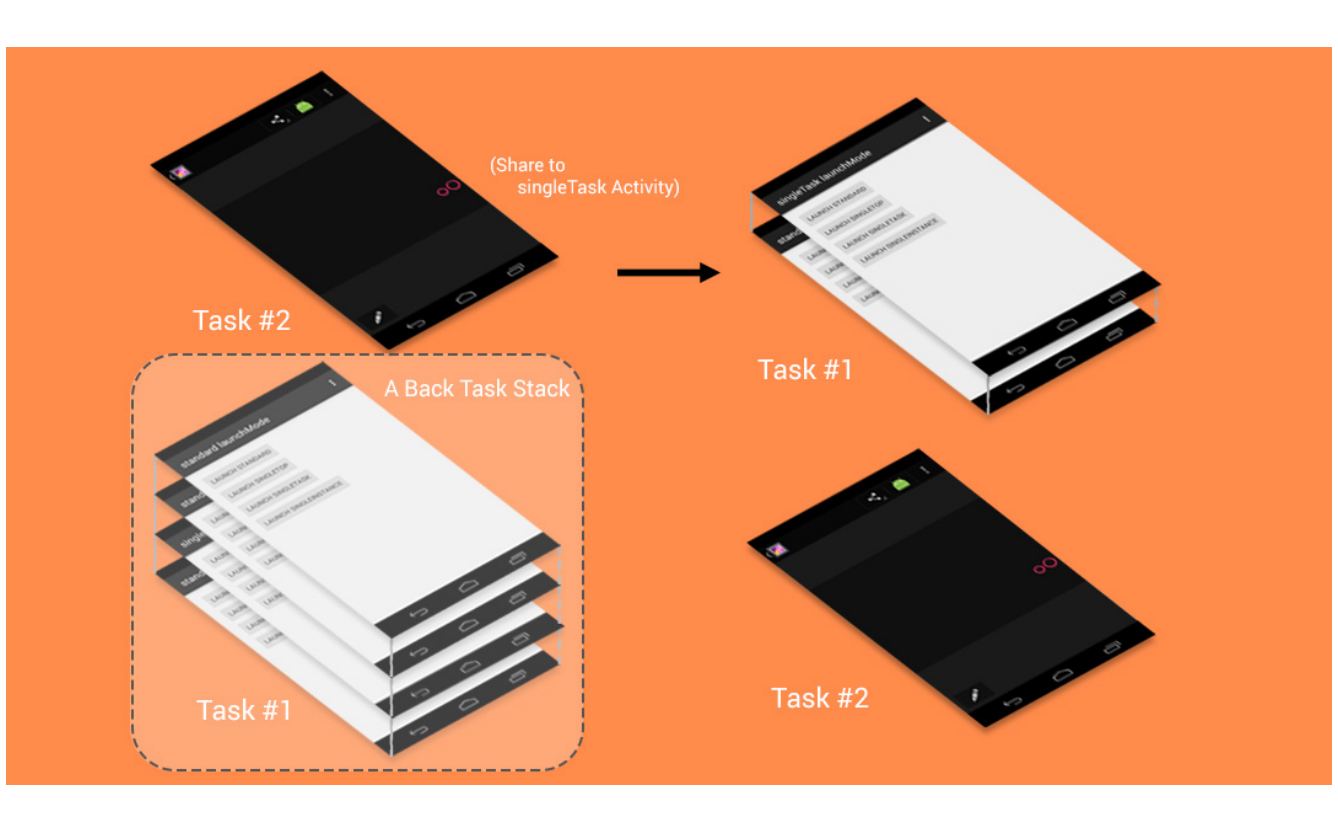




Если не существует задачи, которая бы являлась владельцем вызывающей singleTask Activity, вместо нее будет выведено наверх новое Activity.



В случае, если в какой-либо задаче существует экземпляр Activity, вся задача будет перемещена вверх, и для каждого отдельного Activity, расположенного над singleTask Activity, будет завершен жизненный цикл. Если нажата кнопка «Назад», пользователь должен пройти через Activity в стеке, прежде чем вернуться к вызывающей задаче.



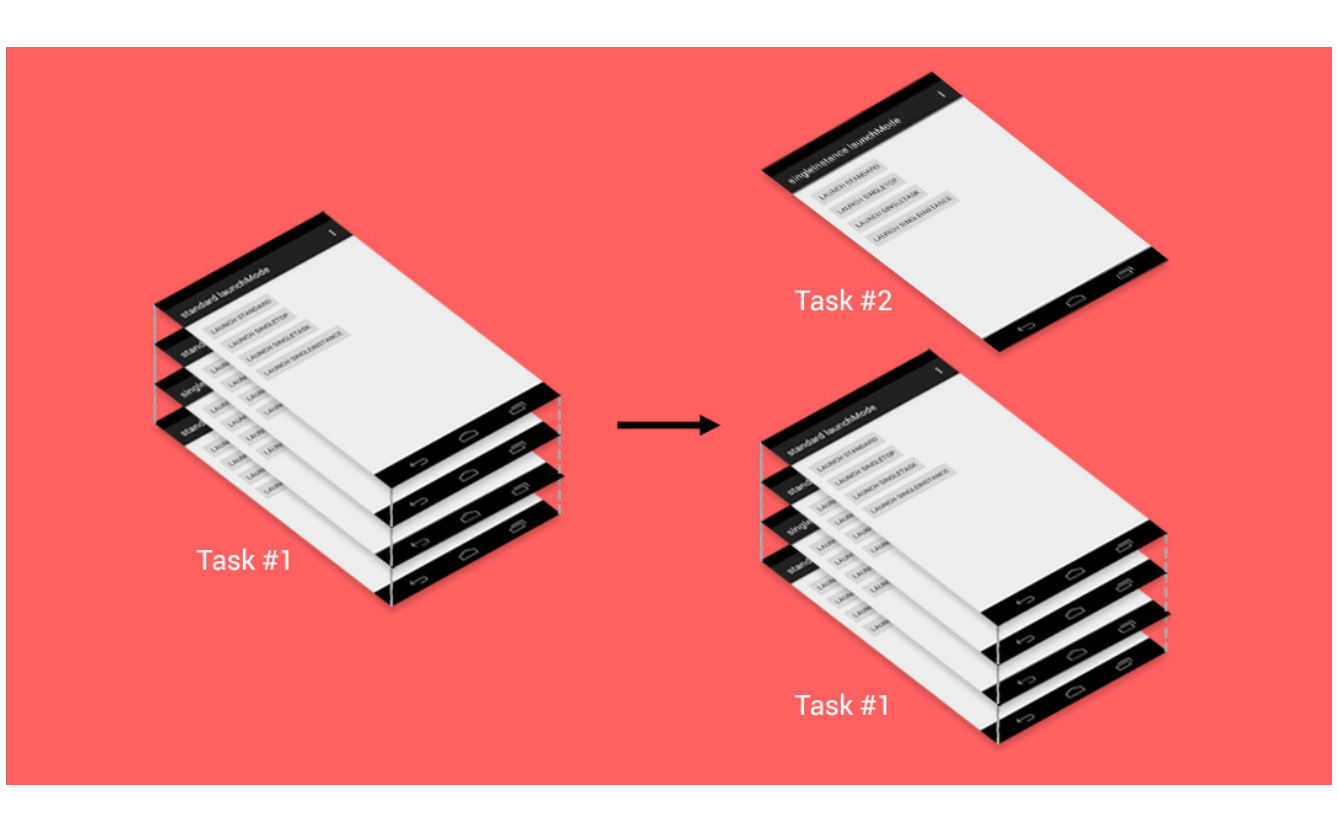
Примером использования этого режима является любое Entry Point Activity, например, страница «Входящие» почтового клиента или таймлайн соцсети. Эти Activity не предполагают более чем одного экземпляра, поэтому singleTask отлично справится со своей задачей. В любом случае вы должны использовать этот режим с умом, так как в этом режиме Activity могут быть уничтожены без подтверждения пользователя, как описано выше.

singleInstance

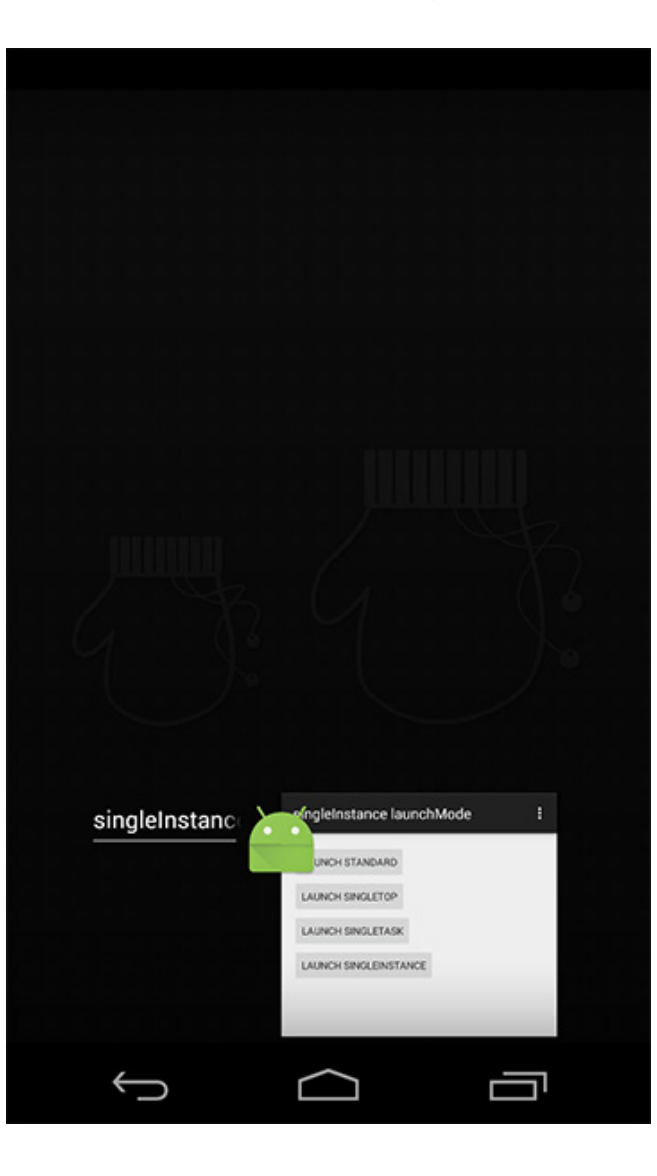
Этот режим очень похож на singleTask, где в системе мог существовать только один экземпляр Activity. **Разница в том, что задача, которая располагает этим Activity, может иметь только одно Activity — то, у которого атрибут singleInstance**. Если из этого вида Activity вызывается другое Activity, автоматически создается новое задание для размещения этого нового Activity. Аналогичным образом, если вызывается singleInstance Activity, будет создана новая задача для размещения этого Activity.

В любом случае результат довольно странный. Из информации, предоставленной `dumpsys`, видно, что в системе есть две задачи, но в диспетчере задач появляется только одна, в зависимости от того, какая из них находится сверху. В результате, хотя есть задача, которая все еще работает в фоновом режиме, мы не можем переключить ее обратно на передний план. Это не имеет вообще никакого смысла.

Вот что происходит, когда вызывается singleInstance Activity, в то время как в стеке уже существует какое-либо Activity.



А вот что мы видим в диспетчере задач.

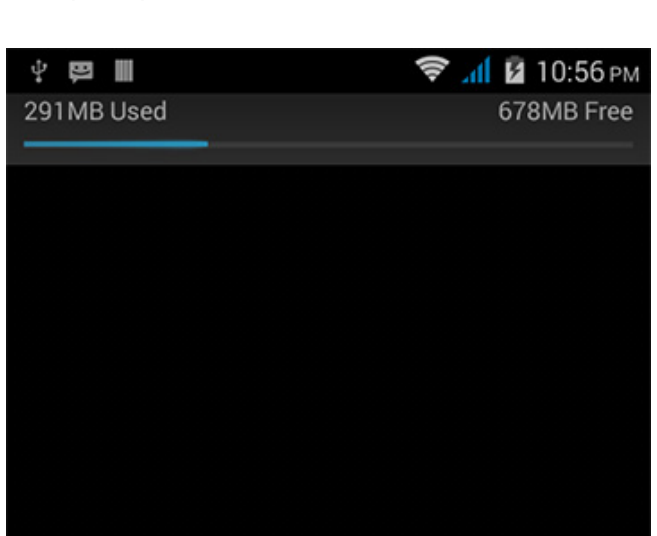


Поскольку эта задача может иметь только одно Activity, мы больше не можем переключаться обратно на задачу № 1. Единственный способ сделать это — перезапустить приложение из лаунчера, но, как в итоге получится, singleInstance задача будет скрыта в фоновом режиме.

Во всяком случае, есть некоторые обходные пути для этой проблемы. Как и в случае с singleTask Activity, просто назначьте атрибут `taskAffinity` для singleInstance Activity, разрешающим существование нескольких задач в диспетчере задач.

```
<activity
    android:name=".SingleInstanceActivity"
    android:label="singleInstance launchMode"
    android:launchMode="singleInstance"
    android:taskAffinity="">
```

Теперь картина имеет больше смысла.



ИНФОРМАЦИЯ

Сайт	otus.ru
Дата регистрации	22 марта 2017
Дата основания	1 апреля 2017
Численность	101–200 человек
Местоположение	Россия
Представитель	OTUS

НОВОСТИ

#Otus_мемы #itmemes
вчера в 19:40

Если новичок, но желаете глаголом (и не только) жечь сердца игроков, то эта стать…
вчера в 14:02

Представьте: вы приходите на новую должность, чтобы руководить подразделением по …
4 марта

Почему лучше изучать NoSQL на курсах с преподавателями, а не самостоятельно? Изучени…
4 марта

Встречаем март вебинарами цифровой Конференции OTUS 🧡 Если вы хотите узнать …
4 марта

2 марта

📊 Какие у вас планы на выходные? ⛅
1 марта

Что вы собираетесь делать в первые выходные весны? ⛅ Планируете ли вы устроить …
1 марта

Хотите научиться придумывать интересные истории для игр? ⭐ Дарья Римарович, р…
1 марта

Представьте. Вы – delivery-менеджер в сфере геймдева. У вас небольшой отдел из 30 про…
29 февраля

ВИДЖЕТ

OTUS News

канал про IT
и онлайн-образование
в сфере информационных технологий

Самые актуальные новости тут

Подписаться

ВКОНТАКТЕ

OTUS. Онлайн-образование

20 154 подписчика

Подписаться на новости

БЛОГ НА ХАБРЕ

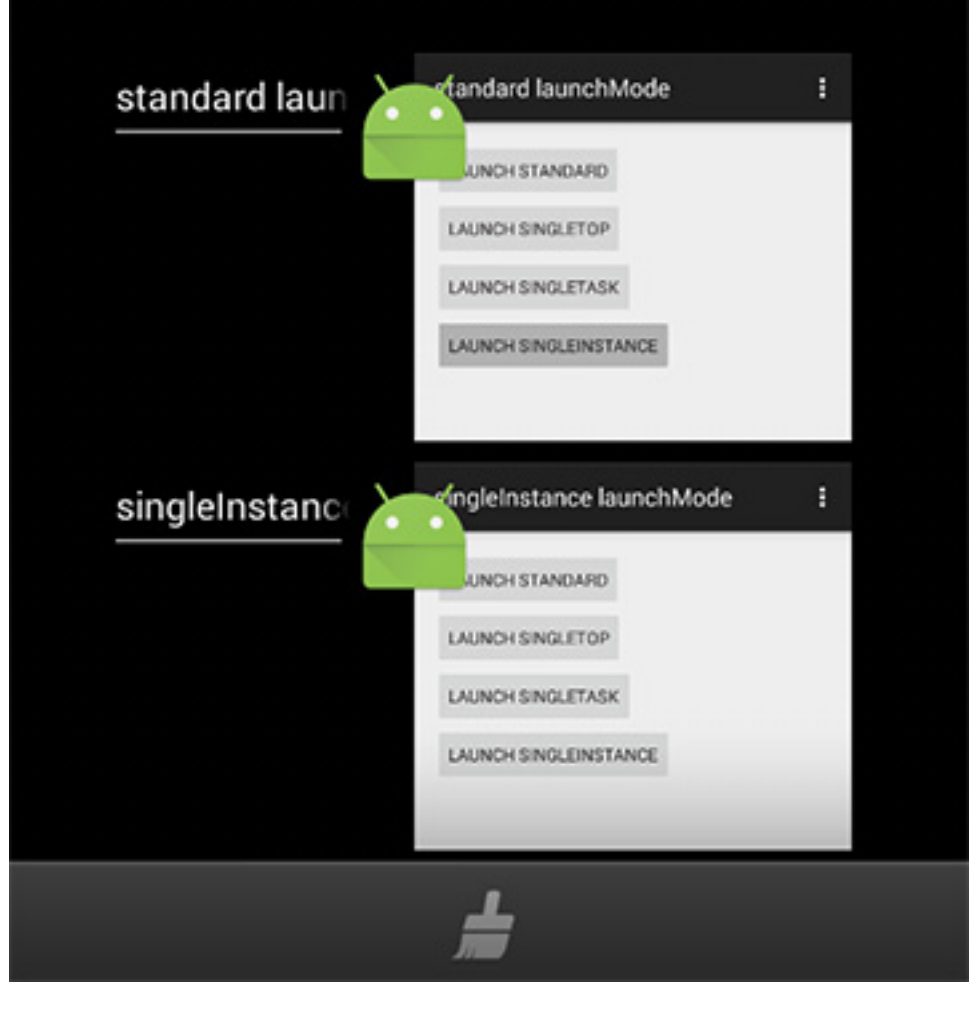
13 часов назад
В общих словах про ЯП Dart
1.9K 4 +4

22 часа назад
Обзор балансировщика traefik
3.3K 4 +4

4 мар в 21:41
Мульти-тенант в Django
2.6K 0

4 мар в 15:46
Основные возможности кластеризации Patroni в PostgreSQL
2.5K 1 +1

4 мар в 12:15
Методы очистки данных в Pandas
3.3K 1 +1



Этот режим используется редко. Некоторые из вариантов использования на практике — это лаунчер-Activity или приложение, для которого вы на 100% уверены, что там должно быть только одно Activity. В любом случае, я предлагаю вам не использовать этот режим, если на то нет крайней необходимости.

Intent-флаги

Помимо назначения режима запуска непосредственно в `AndroidManifest.xml`, мы также можем регулировать поведение с помощью инструмента, называемого **Intent-флагами**, например:

```
Intent intent = new Intent(StandardActivity.this, StandardActivity.class);
intent.addFlags(Intent.FLAG_ACTIVITY_SINGLE_TOP);
startActivity(intent);
```

запустит `StandardActivity` с условием singleTop launchMode.

Есть довольно много флагов, с которыми вы можете работать. Вы можете найти больше информации об этом здесь.

Надеюсь, вы нашли эту статью полезной =)

Узнать подробнее о курсе

Теги: android, activity, launchMode

Хабы: Блог компании OTUS, Разработка под Android

+7 79 6

Редакторский дайджест
Присылаем лучшие статьи раз в месяц

Электропочта

OTUS
Цифровые навыки от ведущих экспертов
Сайт ВКонтакте Telegram

92 Карма 36.4 Рейтинг

OTUS @MaxRokatansky
Редактор

Комментарии 6

ivanrt 26 мар 2020 в 17:33

Помню разбирался с launchMode с документацией, исходниками и методом проб и ошибок. И мне кажется в итоге сделал всё криво. Спасибо за статью.

Ответить

Nashev 2 сен 2022 в 19:54

чё-то нифига в итоге не понял, кроме того что "там всё криво и нормально этим воспользоваться нельзя"...

Ответить

Nashev 9 мая 2023 в 16:35

А для начала - например, вообще непонятно, что такое тут Activity... Пришлось найти https://developer.android.com/reference/android/app/Activity.html и сначала читать его, а ещё раньше (собственно, там и нашёл), вкурить статьи из серии Как работает Android

Ответить

Nashev 10 мая 2023 в 08:29

Кажется, атрибут android:allowTaskReparenting в тегах application или activity в манифесте, радикально влияет на описываемое поведение и просто обязан был быть тут упомянут.

> Whether the activity can move from the task that started it to the task it has an affinity for when that task is next brought to the front. It's `"true"` if it can move, and `"false"` if it remains with the task where it started.
>
> If this attribute isn't set, the value set by the corresponding `allowTaskReparenting` attribute of the `<application>` element applies to the activity. The default value is `"false"`.
>
> Normally, when an activity is started it's associated with the task of the activity that started it and it stays there for its entire lifetime. You can use this attribute to force it to be re-parented to the task it has an affinity for when its current task is no longer displayed. Typically, this is used to cause the activities of an application to move to the main task associated with that application.
>
> For example, if an email message contains a link to a web page, clicking the link brings up an activity that can display the page. That activity is defined by the browser application but is launched as part of the email task. If it's reparented to the browser task, it shows when the browser next comes to the front and is absent when the email task again comes forward.
>
> The affinity of an activity is defined by the `taskAffinity` attribute. The affinity of a task is determined by reading the affinity of its root activity. Therefore, by definition, a root activity is always in a task with the same affinity. Since activities with `"singleTask"` or `"singleInstance"` launch modes can only be at the root of a task, re-parenting is limited to the `"standard"` and `"singleTop"` modes. (See also the `launchMode` attribute.)

Ответить

Nashev 10 мая 2023 в 09:08

Так же в тему атрибут android:documentLaunchMode:

> Specifies how a new instance of an activity is added to a task each time it is launched. This attribute permits the user to have multiple documents from the same application appear in the Recents screen.
>
> This attribute has four values, which produce the following effects when the user opens a document with the application:
>
> `"intoExisting"` The system searches for a task whose base intent's `ComponentName` and data URI match those of the launching intent. If the system finds such a task, the system clears the task and restarts, with the root activity receiving a call to `onNewIntent(android.content.Intent)`. If the system doesn't find such a task, the system creates a new task.
>
> `"always"` The activity creates a new task for the document, even if the document is already opened. This is the same as setting both the `FLAG_ACTIVITY_NEW_DOCUMENT` and `FLAG_ACTIVITY_MULTIPLE_TASK` flags.
>
> `"none"` The activity doesn't create a new task for the activity. This is the default value, which creates a new task only when `FLAG_ACTIVITY_NEW_TASK` is set. The Recents screen treats the activity as it does by default: it displays a single task for the app, which resumes from whatever activity the user last invoked.
>
> `"never"` The activity isn't launched into a new document even if the intent contains `FLAG_ACTIVITY_NEW_DOCUMENT`. Setting this overrides the behavior of the `FLAG_ACTIVITY_NEW_DOCUMENT` and `FLAG_ACTIVITY_MULTIPLE_TASK` flags, if either of these are set in the activity, and the Recents screen displays a single task for the app, which resumes from whatever activity the user last invoked.
>
> **Note:** For values other than `"none"` and `"never"`, the activity is defined with `launchMode="standard"`. If this attribute isn't specified, `documentLaunchMode="none"` is used.

Ответить

Nashev 10 мая 2023 в 09:15

Ну и ссылка на android:launchMode тут просто обязана была быть...

Ответить

Только полноправные пользователи могут оставлять комментарии. Войдите, пожалуйста.

Публикации

ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

Catx2 19 часов назад
Рога переходного периода (из ниоткуда в никуда)
26 мин 5.9K
Ретроспектива
+66 38 33 +33

dmitriyrudnev 22 часа назад
Методика D.I.Y. проекта. Стратегия выживания
Простой 9 мин 4.6K
Обзор
+59 42 13 +13

MaFrance351 23 часа назад
ПИУ-2. Оживляем плазменный текстовый индикатор из СССР
Средний 17 мин 4.2K
Туториал
+53 20 27 +27

dalerank 18 часов назад
Бобры-п[р]огромисты
Простой 4 мин 4.6K
Ретроспектива
+42 21 12 +12

CodeDroidX 18 часов назад
Анонимная P2P-база знаний, где никто не может подделать чужое авторство | Магия асимметричных ключей на практике
Средний 31 мин 3.6K
Туториал
+40 68 22 +22

romanova_ase 19 часов назад
Деплоим сайт в облако за рубль в месяц
8 мин 4.5K
Туториал
+25 73 22 +22

artur_speaking 22 часа назад
Как перестать переусложнять и начать жить
Простой 10 мин 8.9K
Из песочницы
+22 60 37 +37

AlbinaSem 21 час назад
Найм хакеров глазами рекрутера: аналитика и советы для пентестеров и хедхантеров
14 мин 3K
Аналитика
+21 34 4 +4

Soro-Kin 23 часа назад
Ключи Supermicro, SSM, VMWare и лекарство от жадности
5 мин 10K
+21 69 10 +10

Grigory_Otrepyev 23 часа назад
Как нанять работника в ИТ, если он реально нужен в 2024 году
Сложный 9 мин 7.3K
Мнение Recovery Mode
+19 24 8 +8

Показать еще

© 2006–2024, Habr